

AWS Cloud Services Guide

Theory, Manual Setup, and Q&A for EC2, Lambda, SageMaker, and IAM

Part 1: AWS Cloud Services - Theory & Setup

■ ■ Module 1: Amazon EC2 (Elastic Compute Cloud)

1.1 Core Theory

Amazon EC2 provides secure, resizable compute capacity in the cloud. It is the virtual machine (VM) hosting service of AWS.

- **AMI (Amazon Machine Image):** A pre-configured template containing the OS, application server, and applications needed to launch your instance.
- **Instance Families:**
 - *Compute Optimized (C):* For high CPU workloads (ML inference).
 - *Memory Optimized (R):* High RAM (large PySpark memory caches).
 - *Accelerated Computing (G/P):* Equipped with GPUs (TensorFlow/PyTorch training).
- **Security Groups:** Virtual firewalls controlling inbound and outbound network traffic to your instance.

1.2 Step-by-Step Launch & Setup

Deploying a FastAPI backend or Streamlit dashboard to EC2.

Option A: AWS Management Console

1. Navigate to the **EC2 Dashboard** and click **Launch instance**.
2. Name your instance (e.g., **fastapi-backend-server**).
3. Under **Application and OS Images**, select **Ubuntu Server 22.04 LTS**.
4. Choose an **Instance Type** (e.g., **t3.medium** for 2 vCPUs and 4GB RAM).
5. Under **Key pair**, select or create a new key pair (**.pem**) to securely connect via SSH.
6. Under **Network settings**, check **Create security group**. Allow **SSH (Port 22)** from your IP, and add rule to allow **HTTP (Port 80)** and **Custom TCP (Port 8000)** for FastAPI.
7. Click **Launch instance**.

Option B: AWS CLI

```
# Launch a t3.medium Ubuntu Instance
aws ec2 run-instances \
  --image-id ami-0c7217cdde317cfec \
  --count 1 \
  --instance-type t3.medium \
  --key-name my-key-pair \
  --security-group-ids sg-0123456789abcdef0 \
  --subnet-id subnet-0123456789abcdef0
```

Option C: Deployment Script (Docker & FastAPI)

Once launched, connect via SSH and deploy the app:

```
# 1. Update OS and Install Docker
sudo apt-get update -y
sudo apt-get install -y docker.io
sudo systemctl start docker
sudo systemctl enable docker

# 2. Pull and Run the Dockerized Application
sudo docker run -d -p 8000:8000 sinhakishan/rag-backend:latest
```

■ Module 2: AWS Lambda (Serverless Compute)

2.1 Core Theory

AWS Lambda is a serverless, event-driven compute service that executes code in response to events.

- **Serverless:** No servers to provision or manage. Scales automatically from zero to thousands of active concurrent requests.
- **Pricing Model:** Billed by execution time (in milliseconds) and resource allocation.
- **Cold Start:** The initialization delay that occurs when a Lambda function runs for the first time in a new execution container.

2.2 Step-by-Step Creation & Setup

Building a serverless Python trigger that processes incoming transaction logs.

Option A: AWS Management Console

1. Open the **AWS Lambda Console** and click **Create function**.

2. Select **Author from scratch**. Name it **ProcessTransactionLogs**.
3. Set Runtime to **Python 3.12**.
4. Under **Permissions**, expand the Execution role and select **Create a new role with basic Lambda permissions**.
5. Click **Create function**.
6. Replace the boilerplate code with a custom python handler:

```
import json
import boto3

def lambda_handler(event, context):
    print("Received Event: ", json.dumps(event))
    # Simple logic to log payload to S3
    return {
        'statusCode': 200,
        'body': json.dumps('Transaction processed successfully!')
    }
```

7. Click **Deploy**. To expose as an API, click **Add trigger**, select **API Gateway**, create an **HTTP API**, and set Security to **Open**.

Option B: AWS CLI

```
# Zip the deployment package
zip function.zip lambda_function.py

# Create Lambda function via CLI
aws lambda create-function \
  --function-name ProcessTransactionLogs \
  --zip-file fileb://function.zip \
  --handler lambda_function.lambda_handler \
  --runtime python3.12 \
  --role arn:aws:iam::123456789012:role/lambda-basic-execution-role
```

■ Module 3: AWS SageMaker (Managed ML Platform)

3.1 Core Theory

AWS SageMaker is a fully managed service that provides every developer and data scientist with the ability to build, train, and deploy machine learning models quickly.

- **SageMaker Notebooks**: Managed Jupyter notebook instances.

- **Training Jobs:** Spins up a transient computing cluster (e.g., GPU instances) to execute model training, outputs model artifacts to S3, and shuts down.
- **Real-time Endpoints:** Deploys model artifacts as an auto-scaling REST API.

3.2 Step-by-Step Training & Deployment

Step 1: Launch SageMaker Notebook Instance (Console)

1. Go to the **SageMaker Console**, select **Notebook instances**, and click **Create notebook instance**.
2. Name it **sales-forecasting-notebook**. Select **ml.t3.medium**.
3. Under **IAM role**, create a role allowing SageMaker access to S3. Click **Create notebook instance**.
4. Once active, click **Open JupyterLab**.

Step 2: Training & Deploying using SDK (Python Code)

Execute this inside the Jupyter notebook to train an XGBoost model and deploy the endpoint:

```
import sagemaker
from sagemaker.xgboost.estimator import XGBoost

session = sagemaker.Session()
role = sagemaker.get_execution_role()

# Create XGBoost Estimator
xgb_estimator = XGBoost(
    entry_point="train.py",
    role=role,
    instance_count=1,
    instance_type="ml.m5.xlarge",
    framework_version="1.5-1",
    hyperparameters={"max_depth": 6, "eta": 0.05, "objective": "reg:squarederror"}
)

# Fit model using data in S3
xgb_estimator.fit({"train": "s3://my-bucket/train/", "validation": "s3://my-bucket/val/"})

# Deploy to real-time endpoint
predictor = xgb_estimator.deploy(
    initial_instance_count=1,
    instance_type="ml.t2.medium"
)
```

■ Module 4: AWS IAM (Identity & Access Management)

4.1 Core Theory

IAM controls access to AWS resources securely.

- **Principle of Least Privilege:** Granting users only the minimum permissions required to perform their specific tasks.
- **Roles vs. Users:** Users represent humans with long-term credentials. **Roles** are temporary credentials assumed by AWS services (e.g., EC2) or federated users to perform actions securely.
- **Policies:** JSON documents defining what actions are allowed on what resources.

4.2 Step-by-Step Secure Role Mapping

Step 1: Create a custom IAM Policy (JSON)

Create a policy allowing read/write access strictly to our ML models folder in S3.

1. Open the **IAM Console** -> **Policies** -> **Create policy**.
2. Go to the **JSON** tab and paste:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "s3:GetObject",
        "s3:PutObject"
      ],
      "Resource": "arn:aws:s3:::kishan-ml-bucket/models/*"
    }
  ]
}
```

3. Name it **S3ReadWriteModelsPolicy** and click **Create policy**.

Step 2: Create IAM Role for EC2

1. Go to **IAM Console** -> **Roles** -> **Create role**.
2. Select **AWS service** as trust type, and **EC2** as the service that will use this role.
3. Search for and check **S3ReadWriteModelsPolicy**.
4. Name the role **EC2S3ModelAccessRole** and click **Create role**.

Step 3: Attach Role to EC2 Instance

1. Go to **EC2 Console** -> **Instances** -> Check your instance.
 2. Click **Actions** -> **Security** -> **Modify IAM role**.
 3. Select **EC2S3ReadOnlyAccessRole** and click **Update IAM role**. The instance can now read/write S3 models securely without any embedded keys!
-

Part 2: AWS Q&A

Q1: What is the difference between an IAM User and an IAM Role? When would you use a Role?

Hinglish Explanation (WhatsApp Language):

IAM User ek single person (human) ko represent karta hai jiske paas long-term username aur password ya access keys hoti hain. Lekin **IAM Role** kisi specific user ka nahi hota, ye temporary access provides karta hai.

Jab kisi AWS service (jaise EC2 instance) ko dusri service (jaise S3 bucket) se data read karna ho, toh hum instance mein access keys hardcode karne ki jagah **IAM Role** assign karte hain. Isse security leaks nahi hote.

Formal Answer (English):

- **IAM User:** A permanent identity with long-term credentials (password or access keys) representing an individual user.
 - **IAM Role:** An identity with temporary credentials that has no permanent security keys associated with it. It can be assumed by any authorized entity (users, services, or applications).
 - **Use Case:** Roles are preferred for **service-to-service authorization** (e.g., allowing an EC2 server to read S3 buckets) to eliminate the risk of hardcoded credentials.
-

Q2: What is a Lambda Cold Start? How do you prevent or mitigate it?

Hinglish Explanation (WhatsApp Language):

Cold Start tab hota hai jab Lambda function bohot dino baad ya pehli baar call ho, aur AWS backend ko container spin-up aur modules load karne mein 1-3 seconds ka delay lag jaye.

Isse rokne ke liye hum **Provisioned Concurrency** use karte hain jo container ko ready (warm) rakhta hai. Code optimization se bhi startup delay kam hota hai.

Formal Answer (English):

A **Cold Start** is the latency latency latency that occurs when a Lambda function is invoked for the first time or after a period of inactivity. AWS must provision an execution container, download the code, and initialize the runtime.

- **Mitigation:**

1. **Provisioned Concurrency:** Configures a dedicated number of warm execution environments, eliminating initialization latency.
 2. **Code Optimization:** Minimize import libraries and optimize function package size.
 3. **Memory Allocation:** Allocating more memory automatically allocates more CPU, reducing compilation times.
-

Q3: Detail the difference between SageMaker Notebook Instances and SageMaker Studio.

Hinglish Explanation (WhatsApp Language):

Notebook Instance ek dedicated virtual server (EC2 VM) hota hai jis par Jupyter environment runs hota hai, jo single developers ke liye convenient hai.

SageMaker Studio ek advanced collaborative web IDE hai jisme data pipelines, AutoML, experiment tracking (MLflow), aur dynamic resource scaling built-in unified dashboard mein integrated hain.

Formal Answer (English):

- **SageMaker Notebook Instance:** A single virtual machine running Jupyter. It is configured with pre-selected instance sizes (e.g., **ml.t3.medium**) and has a fixed compute allocation.
 - **SageMaker Studio:** A unified web-based IDE that supports collaborative machine learning lifecycles. It integrates features like SageMaker Pipelines, Model Registry, and Experiment Tracking directly, allowing users to spin up notebook engines dynamically.
-

Q4: How do you choose the right EC2 instance type for model training vs model inference?

Hinglish Explanation (WhatsApp Language):

Model Training ke liye high parallel matrix operations ki zarurat hoti hai, isliye hum **GPU instances (G or P family)** ya memory-rich nodes use karte hain.

Model Inference (predictions) mein parameters change nahi hote, isliye fast sequential execution ke liye **Compute Optimized (C family)** ya lightweight instances (like t3.medium) compute costs bachane ke liye perfect hain.

Formal Answer (English):

- **Model Training:** Highly parallelized floating-point operations. Requires **GPU-accelerated instances (G5, P4 families)** or memory-rich instances to fit large batches of weights during backpropagation.
 - **Model Inference:** Forward-propagation calculation only. Requires lower computing resources. **Compute-optimized instances (C6i, C7g)** or general-purpose instances (T3 family) are selected to balance execution latency and hosting costs.
-